Digital VLSI Design

Assignment 1

4-bit Unsigned Array Multiplier from RTL to GDSII

SkyWater SKY130 (`sky130_fd_sc_hd`) | Icarus Verilog, Yosys, OpenSTA, LibreLane

Team member	**Roll number**
Ande Karthik	2023102009
Kalluri Roshan Lal	2023102010

Contents

1 Overview and Environment **3**
1.1 Tools and environment . . . 3
1.2 Repository layout . . . 3

2 Part A: RTL Design and Verification **4**
2.1 Full adder . . . 4
2.2 Array multiplier architecture . . . 4
2.3 Block diagram . . . 5
2.4 Testbench . . . 6
2.5 Simulation results . . . 6
2.6 Hand-worked example (task 5) . . . 6
2.7 Waveform . . . 7

3 Part B: Logic Synthesis with Yosys **8**
3.1 Synthesis script . . . 8
3.2 Part B results . . . 8
3.3 Full cell breakdown . . . 9
3.4 Gate-level (netlist) simulation . . . 9

4 Part C: Static Timing Analysis with OpenSTA **10**
4.1 STA script and constraints . . . 10
4.2 Finding the maximum frequency . . . 10
4.3 Part C results . . . 11
4.4 Critical path . . . 11
4.5 Comparison with the $2N = 8$ full-adder estimate . . . 11

5 Part D: Physical Design with LibreLane **12**
5.1 Configuration and flow . . . 12
5.2 Timing closure: every attempt . . . 12
5.3 Part D results . . . 13
5.4 Timing at every signoff corner . . . 13
5.5 Post-route F_{max} compared with Part C . . . 13
5.6 Cells added by place and route . . . 14
5.7 Final layout . . . 15

6 Answers to the Report Questions **18**
Q1. Logic cell count . . . 18
Q2. Critical path . . . 18
Q3. Flip-flop area . . . 18
Q4. Post-route Fmax and added cells . . . 18
Q5. DRC and LVS . . . 19
Q6. Scaling to 8 bits . . . 19

1 Overview and Environment

We designed a 4-bit × 4-bit unsigned array multiplier in Verilog, verified it with an exhaustive self-checking testbench, synthesised it to SKY130 standard cells with Yosys, analysed its timing with OpenSTA, and took it through place and route to a GDSII layout with LibreLane. The design registers its inputs and its output, so every internal timing path runs from one flip-flop to another. The latency is 2 clock cycles and the throughput is one product per clock cycle.

1.1 Tools and environment

Stage	Tool and version
RTL and gate-level simulation	Icarus Verilog 14.0 (`iverilog -g2012`, `vvp`)
Waveform viewing	GTKWave 4.0.0
Logic synthesis	Yosys 0.70 with ABC
Static timing analysis	OpenSTA 3.1.0
RTL-to-GDSII flow	LibreLane 3.1.0.dev3, Classic flow (Yosys, OpenROAD, Magic, KLayout, Netgen)
Layout viewing	KLayout with the SKY130 layer properties from the PDK
PDK and library	`sky130A`, `sky130_fd_sc_hd`; Liberty file `sky130_fd_sc_hd__tt_025C_1v80.lib` (typical corner) for Parts B and C
Environment	IIC-OSIC-TOOLS Docker image `hpretl/iic-osic-tools:latest` on Ubuntu 24.04.4; the project folder is mounted at `/foss/designs/mult4_2023102009`

Table 1: Tools used in each stage of the flow.

The container's default PDK is IHP `ihp-sg13g2`, so SKY130 is selected explicitly in every step (`$PDK_ROOT/sky130A/...` paths in Parts A–C and `PDK=sky130A` in Part D).

1.2 Repository layout

Path	Contents
`README.md`	team, environment, every command in order
`rtl/full_adder.v`	full adder (`assign` statements)
`rtl/mult_array.v`	array multiplier: 16 AND gates, 12 `full_adder` instances, 16 flip-flops
`tb/tb_mult.v`	exhaustive self-checking testbench (256 vectors)
`synth/`	`synth.tcl` → `mult_array_netlist.v`, `mult_array_stat.rpt`
`sta/`	`sta.tcl` → `mult_array_timing.rpt`
`pnr/`	`config.json` → `metrics.json`, `mult_array.gds` (final run)
`results/PartA..PartD/`	logs, tables, screenshots and write-ups for every task
`report/report.pdf`	this report

Table 2: Repository layout.

2 Part A: RTL Design and Verification

2.1 Full adder

`rtl/full_adder.v` defines module `full_adder` with inputs `a`, `b`, `cin` and outputs `sum`, `cout`, written only with `assign` statements:

$$\texttt{sum} = a \oplus b \oplus cin \qquad \text{(1 when an odd number of inputs are 1)}$$
$$\texttt{cout} = a\,b + b\,cin + a\,cin \qquad \text{(majority: 1 when two or more inputs are 1)}$$

2.2 Array multiplier architecture

For every bit b_i of B there is one partial product $pp_i = A \wedge b_i$, so $pp_i[j] = a[j] \cdot b[i]$. This gives $4 \times 4 = 16$ AND gates. Row 0 has no adders. Rows 1 to 3 are 4-bit ripple-carry adders of 4 full adders each, giving $3 \times 4 = 12$ full adders. Each row adds its partial product to the 4 bits passed down from the row above; the lowest sum bit of each row is a finished product bit.

Row	**Inputs to full adder j**	**Outputs**
0	no adders (AND gates only)	$p[0] = pp_0[0]$; $r_0 = \{0, pp_0[3], pp_0[2], pp_0[1]\}$
1	$pp_1[j], r_0[j], c_1[j]$ $(c_1[0] = 0)$	$s_1[j], c_1[j{+}1]$; $p[1] = s_1[0]$; $r_1 = \{c_1[4], s_1[3], s_1[2], s_1[1]\}$
2	$pp_2[j], r_1[j], c_2[j]$ $(c_2[0] = 0)$	$s_2[j], c_2[j{+}1]$; $p[2] = s_2[0]$; $r_2 = \{c_2[4], s_2[3], s_2[2], s_2[1]\}$
3	$pp_3[j], r_2[j], c_3[j]$ $(c_3[0] = 0)$	$s_3[j], c_3[j{+}1]$; $p[3] = s_3[0]$; $p[7{:}4] = \{c_3[4], s_3[3], s_3[2], s_3[1]\}$

Table 3: Connections of the array (Section 2 of the assignment).

Implementation in `rtl/mult_array.v`:

- Ports `clk`, `rst_n` (asynchronous, active low), `a[3:0]`, `b[3:0]`, `p[7:0]` (registered), exactly as specified.
- Input registers `a_reg`, `b_reg` (8 flip-flops) capture `a` and `b` on every rising edge; reset clears them to 0.
- Partial products `ppi = a_reg & {4{b_reg[i]}}`: the 16 AND gates.
- Three adder rows with 4 `full_adder` instances each (`fa10`..`fa33`), 12 instances in total; carry chains `c1`, `c2`, `c3` with bit 0 tied to 0.
- The 8-bit result is registered into `p` (8 flip-flops), giving 16 flip-flops in total.
- The `*` operator is not used anywhere in the RTL.

2.3 Block diagram

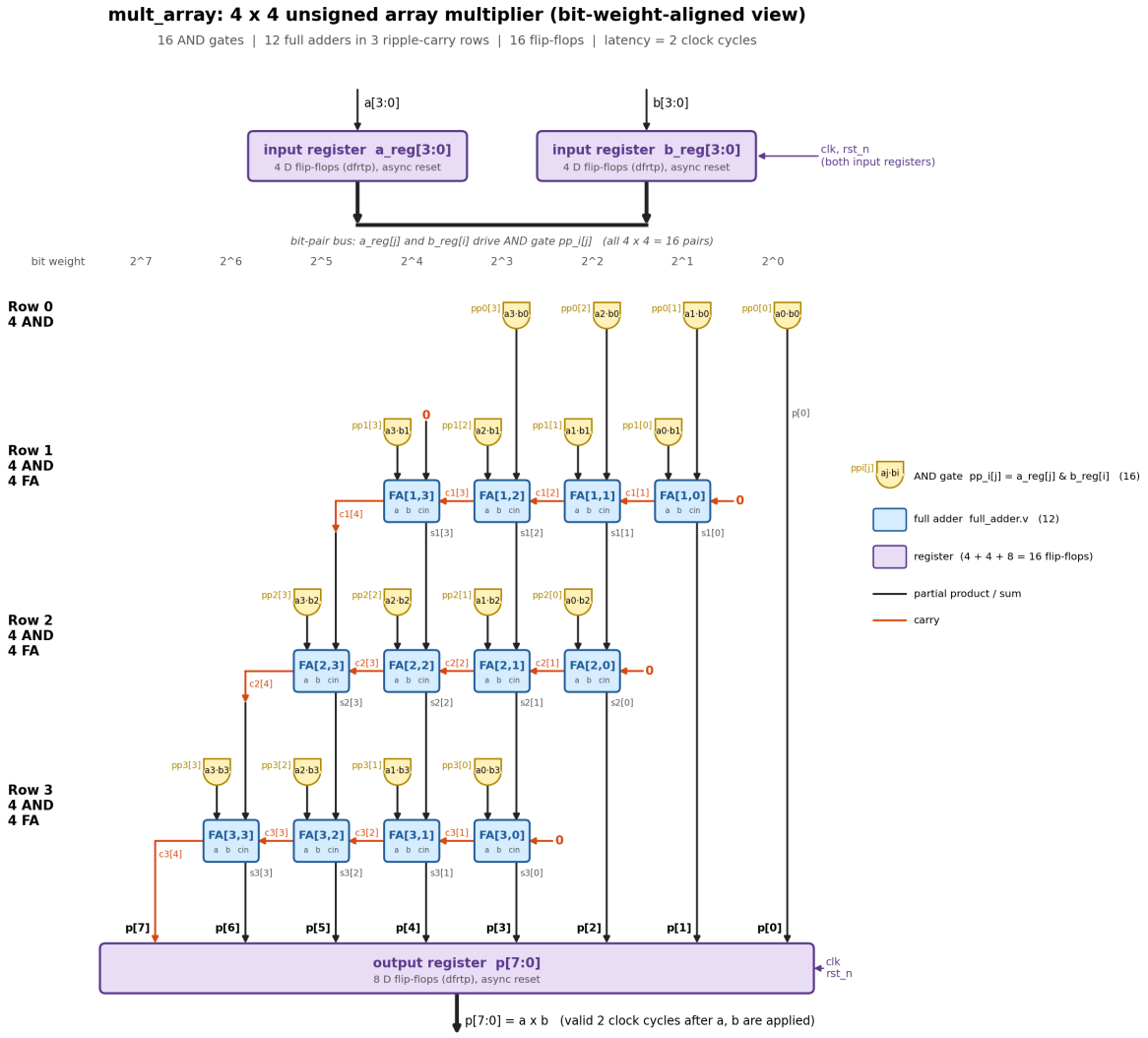


Figure 1: Block diagram of `mult_array` in a bit-weight-aligned layout: input registers `a_reg`/`b_reg` (8 flip-flops), 16 AND gates $pp_i[j]$, 12 full adders FA$[i,j]$ in three ripple-carry rows, and the output register `p` (8 flip-flops). Partial products and sums flow straight down, carries flow from right to left.

2.4 Testbench

Requirement	How `tb/tb_mult.v` meets it
10 ns clock; `rst_n` low for the first 2 cycles	`always #5 clk = ~clk`. `rst_n` is 0 from time 0 and is released at the falling edge after the second rising edge (20 ns).
All 256 input combinations	Nested loops `a_value` = 0..15 and `b_value` = 0..15.
Wait for the 2-cycle latency and compare with `a * b`	Inputs change at a falling edge; the testbench waits two rising edges (capture into `a_reg`/`b_reg`, then into `p`) and checks `p` 2 ns later against `expected = a_value * b_value`. The `*` operator appears only here.
Print every mismatch; one summary line	`Mismatch: a=.. b=.. expected=.. got=..` for each error, then exactly one `PASS: 256 vectors checked, 0 errors` or `FAIL: 256 vectors checked, N errors`.
Waveform file `mult.vcd`	`$dumpfile("mult.vcd"); $dumpvars(0, tb_mult);`

Table 4: Testbench requirements and how `tb/tb_mult.v` meets them.

Each vector takes 2 clock cycles (20 ns): vector k is applied at $30 + 20k$ ns and checked 17 ns later, so the last check is at $47 + 20 \times 255 = 5147$ ns, matching `$finish called at 5147000 (1ps)`.

2.5 Simulation results

```
$ iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
$ vvp sim
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
```

Error-injection test (task 6). To check that the testbench catches errors, the sum in `full_adder.v` was temporarily changed from `a ^ b ^ cin` to `a ^ b`, which ignores the carry-in. The testbench was run again and the file was then restored with `git checkout rtl/full_adder.v` (`git diff` empty, PASS again):

```
Mismatch: a=3 b=3 expected=9 got=5
Mismatch: a=3 b=6 expected=18 got=10
Mismatch: a=3 b=7 expected=21 got=9
Mismatch: a=3 b=11 expected=33 got=29
Mismatch: a=3 b=12 expected=36 got=20
... (90 more Mismatch lines)
FAIL: 256 vectors checked, 95 errors
```

95 of the 256 products are wrong. A vector can only fail when some full adder has a carry-in of 1, so products that never create an internal carry (for example $a = 0$, $b = 0$ or a power-of-two operand) still pass. The testbench therefore detects a broken adder. Logs: `results/PartA/1_rtl_sim_pass.log`, `results/PartA/2_fail_test.log`.

2.6 Hand-worked example (task 5)

Roll number 2023102009 (last digit 9): $A = 9 + 5 = 14 = 1110_2$, $B = 7 = 0111_2$ ($A \neq 7$, so B stays 7). The expected product is $14 \times 7 = 98 = 0110\,0010_2$.

```
        1 1 1 0     A  = 14
      x 0 1 1 1     B  =  7
-----------------
        1 1 1 0     pp0 = A AND b0 (b0 = 1)              = 14
      1 1 1 0       pp1 = A AND b1 (b1 = 1), shifted 1   = 28
    1 1 1 0         pp2 = A AND b2 (b2 = 1), shifted 2   = 56
  0 0 0 0           pp3 = A AND b3 (b3 = 0), shifted 3   =  0
-----------------
  0 1 1 0 0 0 1 0   P = 14 + 28 + 56 + 0 = 98
```

The same product through the adder rows of Table 3 (bits written MSB first):

Row	pp_i	r_{i-1}	$\{c_i[4], s_i\}$	**Outputs**
0	1110	–	–	$p[0] = 0$, $r_0 = 0111$
1	1110	0111	1 0101 $(14 + 7 = 21)$	$p[1] = 1$, $r_1 = 1010$
2	1110	1010	1 1000 $(14 + 10 = 24)$	$p[2] = 0$, $r_2 = 1100$
3	0000	1100	0 1100 $(0 + 12 = 12)$	$p[3] = 0$, $p[7{:}4] = 0110$

Table 5: Row-by-row evaluation of 14×7 in the array.

$P = \{p[7{:}4], p[3], p[2], p[1], p[0]\} = \{0110, 0, 0, 1, 0\} = 0110\,0010_2 = 98$, the same as the shift-and-add result. The full working, including every full adder's inputs and outputs, is in `results/PartA/hand_example.txt`.

2.7 Waveform

In the exhaustive testbench the pair $a = 14$, $b = 7$ is vector $14 \times 16 + 7 = 231$, applied at $30 + 20 \times 231 = 4650$ ns.

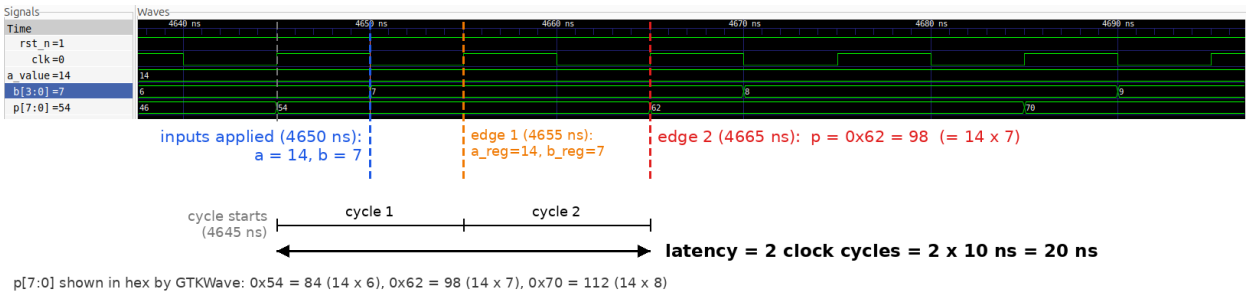


Figure 2: GTKWave view of `clk`, `rst_n`, the input values (`a_value` is the testbench variable that drives `a`), `b` and `p` for $A = 14$, $B = 7$. The inputs are applied at 4650 ns, captured into the input registers at rising edge 1 (4655 ns), and the product `0x62` $= 98$ appears on `p` at rising edge 2 (4665 ns): the latency is 2 clock cycles (20 ns). GTKWave shows `p` in hexadecimal (`0x54` $= 84$, `0x62` $= 98$, `0x70` $= 112$).

3 Part B: Logic Synthesis with Yosys

3.1 Synthesis script

Step	Command in `synth/synth.tcl`	Purpose
1	`yosys read_verilog rtl/full_adder.v rtl/mult_array.v`	Read the design
2	`yosys synth -top mult_array -flatten`	Generic synthesis; flatten the 12 full adders
3	`yosys dfflibmap -liberty $lib`	Map flip-flops to SKY130 cells
4	`yosys abc -liberty $lib`	Map the logic to SKY130 gates
5	`yosys opt_clean -purge`	Remove unused wires
6	`yosys tee -o synth/mult_array_stat.rpt stat -liberty $lib`	Cell and area report
7	`yosys write_verilog -noattr synth/mult_array_netlist.v`	Gate-level netlist

Table 6: Steps of `synth/synth.tcl`, run with `yosys -c synth/synth.tcl`.

`$lib` is the typical-corner Liberty file `$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib`.

3.2 Part B results

Metric	Value
Total number of cells	73
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`: D flip-flop with active-low asynchronous reset)
Total cell area (μm^2)	805.77
Flip-flop area (μm^2)	$400.38 = 16 \times 25.024$
Flip-flop area (% of total)	49.69 %
The 3 most-used cell types and their counts	`dfrtp_1` $\times$ 16, `nand2_1` $\times$ 11, `a21oi_1` $\times$ 4
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 7: Part B results table (from `synth/mult_array_stat.rpt`).

3.3 Full cell breakdown

Group	Cells (sky130_fd_sc_hd__...)	Count	Area (μm^2)
Flip-flops	dfrtp_1 ×16	16	400.384
Sum logic (XOR/XNOR)	xnor2_1 ×4, xnor3_1 ×2, xor3_1 ×1	7	103.850
Carry logic (majority)	maj3_1 ×1	1	10.010
AND gates	and2_0 ×4, and3_1 ×1, and4_1 ×1	6	40.038
NAND / NOR	nand2_1 ×11, nand3_1, nand4_1 ×4, nor2_1, nor2b_1, nor3_1	19	86.334
AND-OR / OR-AND compound	a21oi_1 ×4, a21boi_0 ×2, o21ai_0 ×2, o31a_1 ×2, a21o_1, a22oi_1, a32oi_1, o2111a_1, o2111ai_1, o21a_1, o22ai_1, o311ai_0, o31ai_1, or3_1	20	148.651
Isolation-type cells	lpflow_inputiso1p_1, lpflow_isobufsrc_1 ×2	3	18.768
Inverters	clkinv_1	1	3.754
Total (31 cell types)		**73**	**805.773**

Table 8: All cells in the synthesised netlist, grouped by function (cells without a count appear once).

3.4 Gate-level (netlist) simulation

The synthesised netlist was simulated with the same testbench and the SKY130 cell models, to show that synthesis did not change the behaviour:

```
$ CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
$ iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
    tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
$ vvp gls
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:73: $finish called at 5147000 (1ps)
```

4 Part C: Static Timing Analysis with OpenSTA

4.1 STA script and constraints

Step	Command in `sta/sta.tcl`	Purpose
1	`read_liberty $lib` (tt_025C_1v80)	Read the cell timing library
2	`read_verilog synth/mult_array_netlist.v; link_design mult_array`	Read and link the netlist
3	`create_clock -name clk -period $clock_period [get_ports clk]`	Clock with period T (10 ns, or `STA_PERIOD`)
4	`set_input_delay 1.0 -clock clk [delete_from_list [all_inputs] [get_ports clk]]`	Input arrival time
5	`set_output_delay 1.0 -clock clk [all_outputs]`	Output required time
6	`report_worst_slack -max`	Worst setup slack
7	`report_checks -path_delay max -digits 3 > sta/mult_array_timing.rpt`	Critical path report

Table 9: Steps of `sta/sta.tcl`, run with `sta -exit sta/sta.tcl`.

The analysis is pre-layout: the clock is ideal (no skew or insertion delay) and there is no wire delay.

4.2 Finding the maximum frequency

```
$ sta -exit sta/sta.tcl                    (T = 10.00 ns)
worst slack max 7.40                       (report: +7.404 ns)

T_min = T - WNS   = 10.000 - 7.404 = 2.596 ns
F_max = 1000/T_min = 1000 / 2.596   = 385.21 MHz

$ STA_PERIOD=2.60 sta -exit sta/sta.tcl    (T = T_min rounded to 0.01 ns)
worst slack max 0.00                       (report: +0.004 ns)
```

At $T = 2.60$ ns the slack is +0.004 ns, effectively zero, which confirms T_{min}. The small positive value comes from rounding 2.596 ns up to 2.60 ns.

4.3 Part C results

Metric	Value
Worst slack at $T = 10$ ns (ns)	+7.404
T_{min} (ns)	2.596
F_{max} (MHz)	385.21
Slack at $T = T_{min}$ (ns)	+0.004 at $T = 2.60$ ns (≈ 0)
Critical path start point	`_118_`: input register `b_reg[0]` (`dfrtp_1`, CLK→Q)
Critical path end point	`_112_`: output register `p[6]` (`dfrtp_1`, D = `fa33.sum`)
Number of cells on the critical path	10: 2 flip-flops + 8 combinational cells
Latency (ns) = $2 \times T_{min}$	5.192
Throughput (million results per second)	385.21 (one result per clock cycle)

Table 10: Part C results table (typical corner, pre-layout).

4.4 Critical path

#	Instance	Cell	Function in the multiplier	Incr (ns)	Arrival (ns)
1	`_118_/Q`	`dfrtp_1`	input register `b_reg[0]`, CLK→Q	0.395	0.395
2	`_083_/X`	`and2_0`	AND gate $pp_0[0] = a_reg[0] \cdot b_reg[0]$	0.212	0.607
3	`_055_/X`	`or3_1`	row-1 carry logic (merged)	0.361	0.968
4	`_059_/X`	`and4_1`	row-1 carry logic merged with an `a_reg[1]` AND	0.215	1.183
5	`_070_/X`	`o2111a_1`	carry logic merged with $pp_2[2]$	0.234	1.417
6	`_076_/Y`	`o21ai_0`	carry-chain logic	0.216	1.634
7	`_090_/X`	`xnor3_1`	full-adder sum	0.274	1.907
8	`_091_/X`	`maj3_1`	full-adder carry	0.375	2.282
9	`_092_/X`	`xor3_1`	full-adder sum of FA[3,3] → `fa33.sum`	0.178	2.460
10	`_112_/D`	`dfrtp_1`	output register `p[6]`, setup 0.135 ns	0.000	2.460

Table 11: Critical path from `sta/mult_array_timing.rpt` ($T = 2.60$ ns). Cell roles were traced in `synth/mult_array_netlist.v`.

Data arrival = 2.460 ns; data required = 2.600 − 0.135 = 2.465 ns; slack = +0.004 ns. At $T = 10$ ns the same path has slack 9.865 − 2.460 = +7.404 ns. The input-to-register and register-to-output paths are short, so the register-to-register path through the adder array sets T_{min}. The report also lists the asynchronous reset recovery check (`rst_n` → `RESET_B`); its slack is +9.228 ns at 10 ns and +1.828 ns at 2.60 ns, so it never limits the clock.

4.5 Comparison with the $2N = 8$ full-adder estimate

- The path has 8 combinational cells with 2.065 ns of logic delay (2.460 − 0.395), about 0.26 ns per cell.
- 7 of the 8 cells come from full-adder logic: 3 pure full-adder functions (`xnor3` sum, `maj3` carry, `xor3` sum of FA[3,3]) and 4 compound gates (`or3`, `and4`, `o2111a`, `o21ai`) that implement the ripple carry merged with partial-product ANDs. Only `_083_` (`and2_0`) is a plain AND gate.
- This matches the estimate of about $2N = 8$ full-adder delays well. The match is approximate because `synth -flatten` removes the `full_adder` boundaries and ABC folds AND gates and parts of several full adders into single compound cells.

5 Part D: Physical Design with LibreLane

5.1 Configuration and flow

LibreLane 3.1.0.dev3 (Classic flow) was run inside the IIC-OSIC-TOOLS container with the SKY130 PDK selected (`export PDK=sky130A STD_CELL_LIBRARY=sky130_fd_sc_hd`; `librelane config.json`). The flow runs synthesis (Yosys), floorplan and power grid, tap insertion, placement, clock-tree synthesis and timing repair, routing (OpenROAD), RC extraction and multi-corner STA, DRC (Magic and KLayout), LVS (Netgen), and writes the GDSII.

Listing 1: `pnr/config.json` (final run)

```
{
  "DESIGN_NAME": "mult_array",
  "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"],
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 7.0,
  "FP_SIZING": "absolute",
  "DIE_AREA": [0, 0, 150, 150]
}
```

The die is fixed at 150 μm × 150 μm. The core box is (5.52, 10.88) to (144.44, 138.72) μm, which holds 47 rows of `unithd` sites.

5.2 Timing closure: every attempt

Starting period: $1.2 \times T_{min} = 1.2 \times 2.596 = 3.115$ ns, rounded up to the next 0.5 ns = 3.5 ns. After each run that failed setup timing, CLOCK_PERIOD was increased by 0.5 ns. The configuration and metrics of every attempt are kept in `results/PartD/attempts/`.

Run	Period (ns)	Setup slack (ns)	tt slack (ns)	Hold slack (ns)	DRC	LVS	Result
1	3.5	−2.154	+0.587	+0.100	0	0	FAIL (setup)
2	4.0	−1.654	+1.087	+0.147	0	0	FAIL (setup)
3	4.5	−1.154	+1.587	+0.147	0	0	FAIL (setup)
4	5.0	−0.654	+2.087	+0.147	0	0	FAIL (setup)
5	5.5	−0.155	+2.587	+0.147	0	0	FAIL (setup)
6	6.0	−0.107	+2.899	+0.148	0	0	FAIL (setup)
7	6.5	−0.136	+3.199	+0.141	0	0	FAIL (setup)
8	**7.0**	**+0.364**	+3.699	+0.141	0	0	**PASS**

Table 12: All LibreLane runs. Setup slack = `setup__ws`, the worst over all 9 signoff corners (always the slow corner `ss_100C_1v60`); tt slack = typical corner `nom_tt_025C_1v80`.

From 3.5 ns to 5.5 ns the slack improves by exactly 0.5 ns per step, as expected when only the period changes. At 6.0 ns and 6.5 ns it improves very little and even gets worse. The synthesised netlist was identical in all 8 runs, so the change comes from the place-and-route timing repair choosing a different critical path. The typical corner passed in every run; only the slow corner limited the period.

5.3 Part D results

Metric	Value
CLOCK_PERIOD used (ns)	7.0
Number of runs needed	8 (3.5 → 7.0 ns in 0.5 ns steps)
Core area (μm^2)	17759.5 (die area 22500 = 150 × 150 μm)
Cells after synthesis (from Part B)	73 (16 flip-flops + 57 logic cells)
Cells after place and route	1895 instances (334 excluding decap/fill)
Kinds of cells added	buffers ×22 (`buf_2`, `clkdlybuf4s50_1`, `clkbuf_16`), tap ×245 (`tapvpwrvgnd_1`), decap ×1314 (`decap_*`), fill ×247 (`fill_*`)
Wirelength (μm)	2362 (552 vias)
Setup slack (ns)	+0.364 (worst corner `max_ss_100C_1v60`)
Post-route F_{max} (MHz)	150.70 = 1000 / (7.0 − 0.364)
F_{max} from Part C (MHz)	385.21
DRC errors	0 (Magic 0, KLayout 0, router 0)
LVS errors	0 (Netgen: circuits match uniquely)

Table 13: Part D results table (final run, CLOCK_PERIOD = 7.0 ns, from `pnr/metrics.json`).

5.4 Timing at every signoff corner

Corner (process, V, T)	Wire RC	Setup slack (ns)	Hold slack (ns)	Implied F_{max} (MHz)
tt, 1.80 V, 25 °C	min	3.729	0.363	305.7
	nom	3.699	0.370	302.9
	max	3.678	0.374	301.1
ss, 1.60 V, 100 °C	min	0.444	0.822	152.5
	nom	0.406	0.815	151.7
	max	**0.364**	0.808	**150.7**
ff, 1.95 V, −40 °C	min	4.735	0.141	441.6
	nom	4.726	0.145	439.8
	max	4.719	0.148	438.5

Table 14: Setup and hold slack at each of the 9 signoff corners (final run). Implied F_{max} = 1000 / (7.0 − setup slack).

The worst setup path is register-to-register (`timing__setup_r2r__ws` = 0.364 ns), from `_102_` (`a_reg[0]`) to `_100_` (D = `fa33.sum`, output `p[6]`): the same output bit as the Part C path. Setup clock skew is 0.26–0.28 ns at every corner. Other signoff numbers: 0 antenna violations, 0 max-slew / max-capacitance / max-fanout violations, 0 disconnected pins, worst IR drop 0.77 mV, total power 0.335 mW.

5.5 Post-route F_{max} compared with Part C

Post-route $F_{max} = 1000/(7.0 - 0.364) = 150.70$ MHz, about 2.6 times lower than the 385.21 MHz of Part C. Two effects combine:

1. **Corner** (about 2×). Part C used only the typical corner, while `setup__ws` is set by the slow corner (1.60 V, 100 °C), where the cells are about twice as slow. The same layout at the typical corner gives $1000/(7.0 - 3.699) = 302.9$ MHz.
2. **Physical effects** (about 27 % at the same corner). The minimum period at the typical corner grows from 2.596 ns to 3.301 ns (Table 15).

Component	Part C (ns)	Post-route, tt (ns)	Added (ns)
Launch flip-flop CLK→Q	0.395	0.472	+0.077 (real wire load)
Logic cells + wires	2.065 (8 cells)	2.439 (9 cells)	+0.374 (wire RC, LibreLane's netlist)
Clock uncertainty	0	0.250	+0.250 (LibreLane default)
Clock skew (0.336 − 0.302 − CRPR 0.018)	0	0.016	+0.016
Library setup time	0.135	0.124	−0.011
Minimum period	**2.596**	**3.301**	**+0.705**

Table 15: Where the extra delay comes from at the typical corner (post-route report of path `a_reg[0]` → `p[6]`).

5.6 Cells added by place and route

Part B synthesis gave **73 cells**; after place and route there are **1895 cells** (`design__instance__count`), about 26 times more. The added cells were found by name in the LibreLane cell-frequency report:

Name contains	Cells	Count	Why the flow adds them
`buf`	`buf_2` ×17, `clkdlybuf4s50_1` ×2	19	Timing repair: drive long nets and fix slew, setup and hold.
	`clkbuf_16` ×3	3	Clock-tree synthesis: deliver the clock to the 16 flip-flops with low skew.
`tap`	`tapvpwrvgnd_1`	245	Tie the n-well to VPWR and the substrate to VGND at regular intervals to prevent latch-up.
`decap`	`decap_3` ×130, `decap_4` ×49, `decap_6` ×175, `decap_8` ×55, `decap_12` ×905	1314	Decoupling capacitors between VPWR and VGND that reduce supply noise and IR drop.
`fill`	`fill_1` ×215, `fill_2` ×32	247	Close the gaps so every row is continuous (wells, implants, power rails) and density rules are met.
Added in total		**1828**	
Logic (51) + flip-flops (16), from LibreLane's own synthesis		67	
Total after place and route		**1895**	

Table 16: Instance classes in the final layout.

The logic part is 67 cells rather than Part B's 73 because LibreLane runs its own Yosys synthesis with a different script; $67 + 1828 = 1895$. Without decap and fill cells there are 334 standard-cell instances with an area of 1224.92 μm^2, a utilisation of only 6.9 % of the 17759.5 μm^2 core. The

fixed 150 μm $\times$ 150 μm die is much larger than this design needs, which is why tap, decap and fill cells make up almost all of the instances.

5.7 Final layout

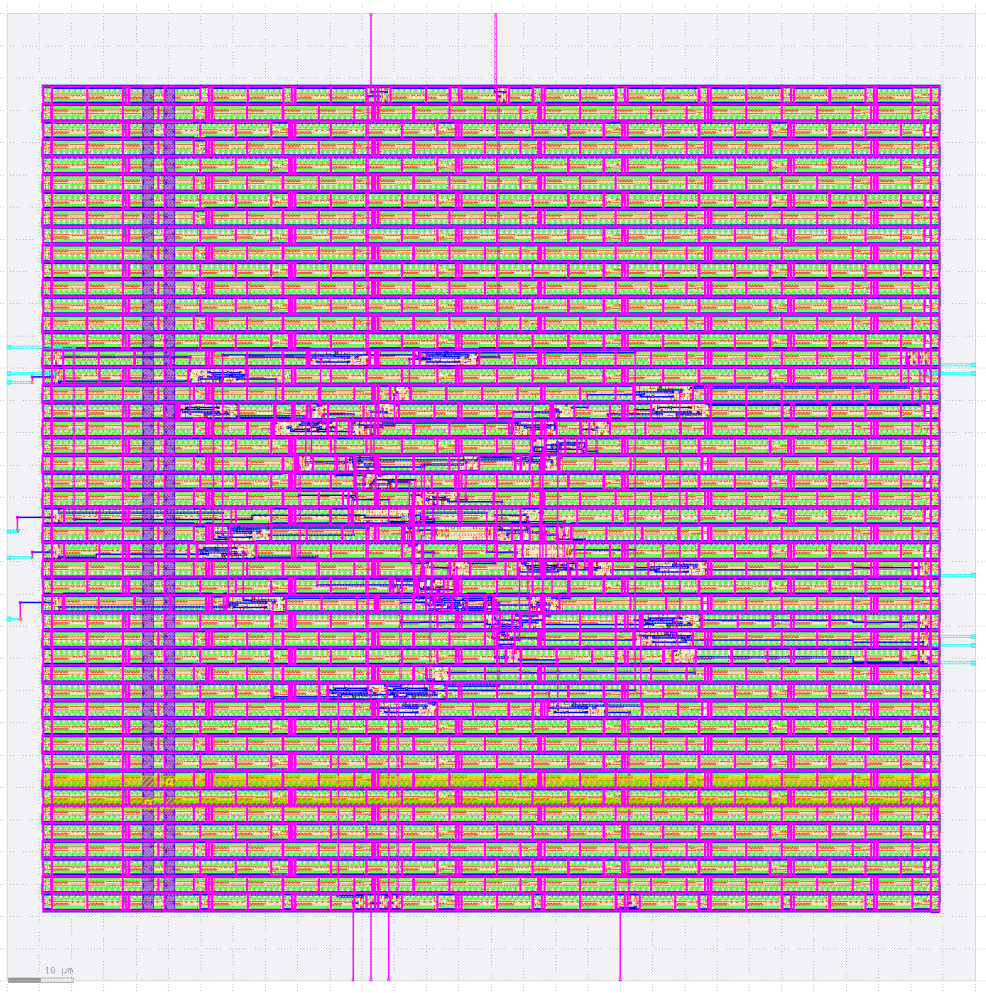


Figure 3: Whole die (150 μm $\times$ 150 μm) in KLayout: I/O pins on the die edges, 47 standard-cell rows with the power grid, and the routed logic in the centre of the core.

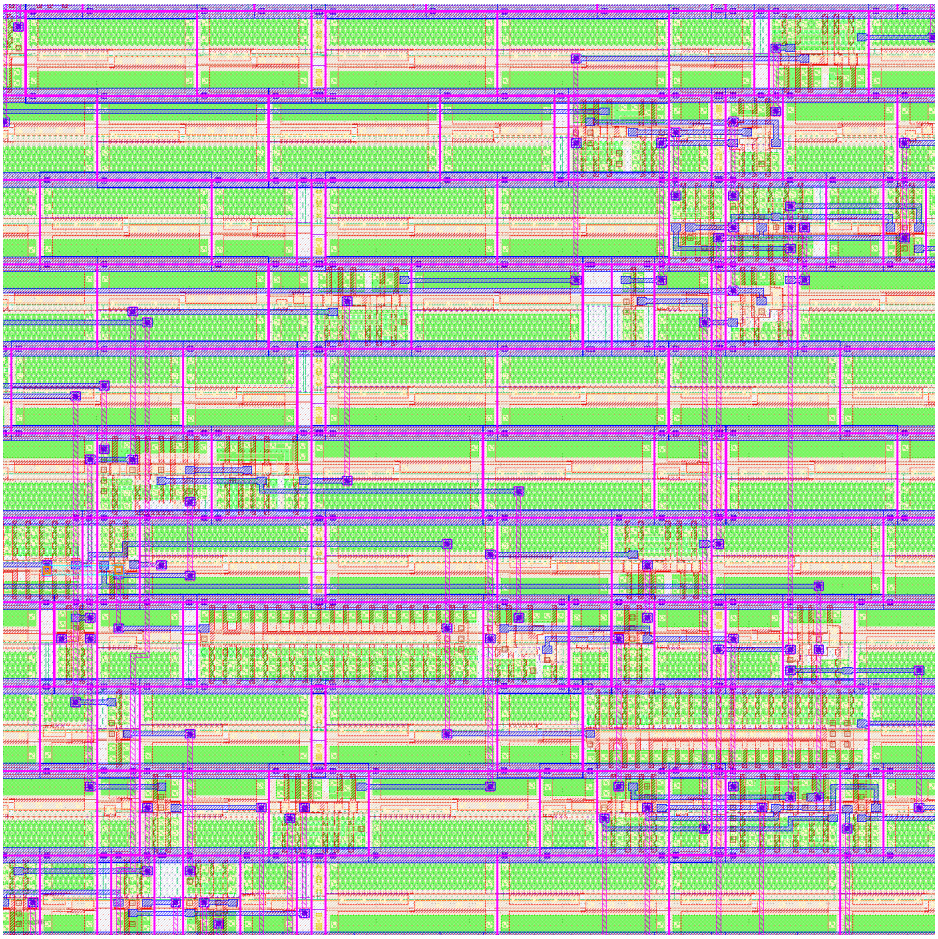


Figure 4: Zoomed view (30 μm $\times$ 30 μm): standard-cell rows bounded by the horizontal power rails, with routed metal-1/metal-2 wires and vias connecting the cell pins.

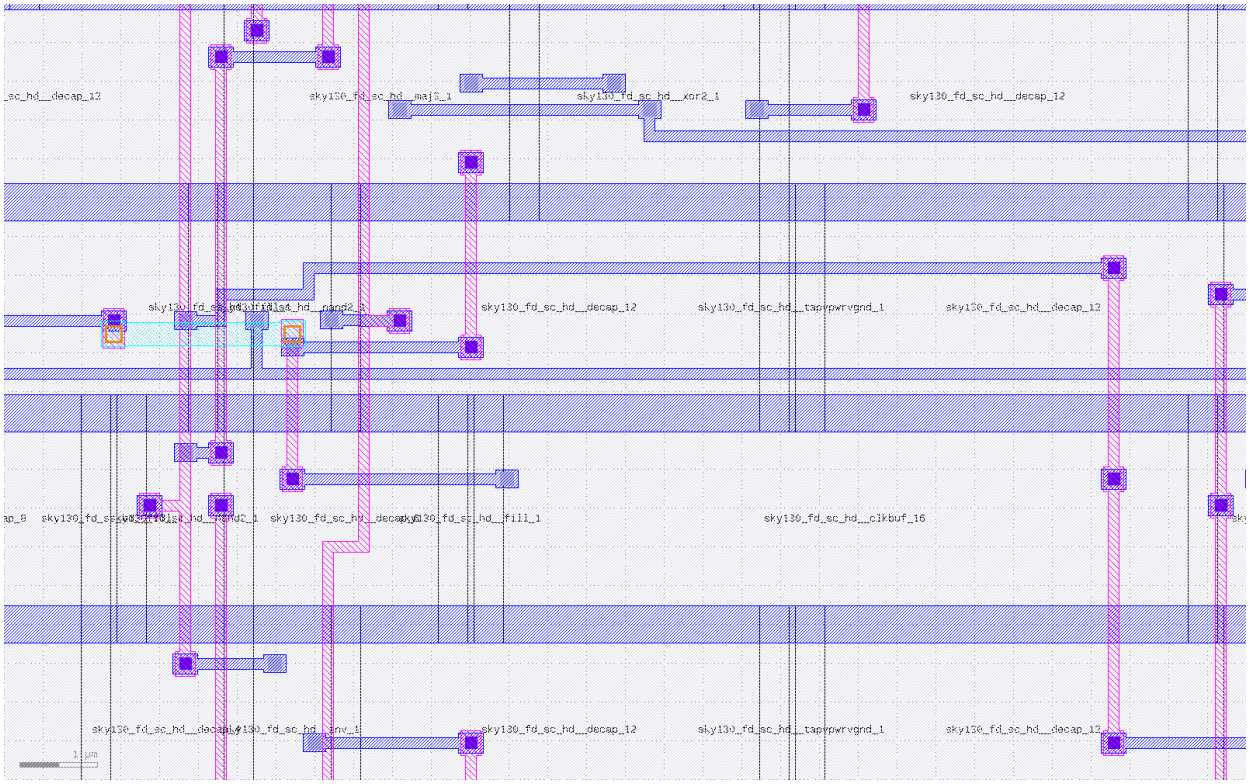


Figure 5: Deeper zoom (16 μm $\times$ 10 μm) with every standard cell drawn as a labelled box: logic cells (`maj3_1`, `xor2_1`, `nand2_1`, `inv_1`) next to cells added by the flow (`decap_12`, `tapvpwrvgnd_1`, `fill_1`, `clkbuf_16`), the power rails between rows and the routed wires.

6 Answers to the Report Questions

Q1. The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic? Explain why the number of cells is not simply 16 + 12.

Synthesis produced 73 cells, of which 16 are flip-flops, so the logic uses **57 cells** (405.39 μm^2) rather than 28. An "AND gate" and a "full adder" are RTL building blocks, not one library cell each. With `-flatten`, Yosys removes the `full_adder` boundaries, and ABC re-implements the whole Boolean function with whichever SKY130 cells give a good area/delay result (Table 8):

- **A full adder needs at least two cells.** Its sum becomes an XOR/XNOR cell (`xnor2`, `xnor3`, `xor3`; 7 in total) and its carry a `maj3` or an equivalent NAND/AOI network. ABC cannot use the two-output SKY130 full-adder cell `fa_1` (the log reports "Detected 9 multi-output cells"), so 12 full adders alone need about 24 or more cells.
- **AND gates are merged into other logic.** Most partial-product ANDs are absorbed into compound gates such as `a21oi`, `a21boi`, `o2111a`, `nand4`; only 6 remain as plain `and2/3/4` cells.
- **Constant inputs simplify some adders.** Every row's carry-in and the top bit of r_0 are 0, so those full adders reduce to half adders built from fewer, simpler gates.

The exact count depends on the RTL style and the tool version (ABC is a heuristic), but it is always well above 28.

Q2. What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about 2N full-adder delays?

The critical path starts at the input register `b_reg[0]` (`_118_`, `dfrtp_1`) and ends at the output register `p[6]` (`_112_`, `dfrtp_1`), whose D input is the sum of the last full adder FA[3,3]. It has **10 cells**: the launching flip-flop, 8 combinational cells (`and2` → `or3` → `and4` → `o2111a` → `o21ai` → `xnor3` → `maj3` → `xor3`) and the capturing flip-flop. The data arrives at 2.460 ns, giving T_{min} = 2.596 ns. The path follows the carry through the adder rows into product bit 6. For $N = 4$ the estimate is $2N = 8$ full-adder delays, which matches the 8 combinational cells; 7 of them come from full-adder logic and the average delay is about 0.26 ns per cell.

Q3. What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?

Flip-flops take 400.38 of 805.77 μm^2 = **49.69 %** of the cell area, although they are only 16 of the 73 cells (22 %). One `dfrtp_1` is 25.02 μm^2: 6.7 times a `nand2_1` (3.75 μm^2), about 3.5 times the average logic cell (7.1 μm^2), and larger than the biggest logic cell in the design (`xor3_1`, 23.77 μm^2). An edge-triggered flip-flop with asynchronous reset is two latches plus clock and reset circuitry, a few dozen transistors compared with 4 for a NAND2. The second reason is scaling: the array logic grows as N^2 (16 ANDs, 12 adders), while the registers grow only as $4N$ (16 flip-flops). At $N = 4$ the logic is still small, so the registers dominate; at 8 bits the flip-flop share falls to about 32 % (Q6).

Q4. Why is the post-route F_{max} lower than the F_{max} from Part C? Which kinds of cells did the flow add, and why is each kind needed?

The post-route F_{max} is 150.70 MHz against 385.21 MHz from Part C, about 2.6 times lower:

1. **Corner.** Part C used only the typical library (tt, 1.80 V, 25 °C). LibreLane signs off at 9 corners and reports the worst, the slow corner (ss, 1.60 V, 100 °C, maximum wire RC). The same layout allows 302.9 MHz at the typical corner, so the corner accounts for about a factor of 2.
2. **Real wires.** Part C had no parasitics. After routing, 2362 μm of wire and 552 vias add resistance and capacitance; the logic delay on the critical path grows by 0.374 ns and CLK→Q by 0.077 ns.
3. **Real clock.** Part C used an ideal clock. The routed design has a clock tree with skew and LibreLane's 0.25 ns clock uncertainty, which reduce the time available for setup.
4. **Different netlist.** LibreLane re-synthesises the RTL with its own script, so the post-route critical path (`a_reg[0]` → `p[6]`, 9 logic cells) is not the same as in Part C.

At the same (typical) corner these physical effects add 0.705 ns to the minimum period (Table 15).

The flow added these cells (Table 16):

- **Buffers (22)**: `buf_2`, `clkdlybuf4s50_1` (timing repair: long nets, slew, setup, hold) and `clkbuf_16` (clock tree: low-skew clock to the 16 flip-flops).
- **Tap cells (245)**: bias the n-well and substrate to prevent latch-up.
- **Decap cells (1314)**: on-chip supply decoupling against switching noise.
- **Fill cells (247)**: keep the wells, implants and power rails continuous across row gaps so the layout passes DRC.

Q5. What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?

DRC (design rule check) verifies the geometry of the layout against the foundry's rules for every layer: minimum widths, spacings, enclosures, overlaps, via rules, density and antenna ratios. These rules describe what the fab can print reliably; a violation can cause shorts, opens or very low yield.

LVS (layout versus schematic) extracts the transistors and their connections from the layout and compares them with the intended netlist: the same devices, nets and pins. It confirms that the layout is the circuit we designed, with no missing connection, accidental short or swapped pin.

Each check covers what the other cannot. A layout can be manufacturable (DRC clean) but implement the wrong circuit, or be the right circuit (LVS clean) but impossible to fabricate. A mask set is very expensive and cannot be fixed after tape-out, so both must pass first. Our final run has 0 DRC errors in Magic and KLayout (the router's intermediate violations, 28 → 3 → 3 → 0, were all fixed), 0 LVS errors in Netgen ("Circuits match uniquely") and 0 antenna violations.

Q6. Using the formulas in Section 2, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.

Quantity	Formula	N = 4 (ours)	N = 8 (estimate)	Change
AND gates	N^2	16	64	×4
Full adders	$N(N-1)$	12	56	×4.67
Flip-flops	$2N + 2N$	16	32	×2
Critical path	$\approx 2N$ full-adder delays	8	16	×2

Table 17: Scaling from 4 to 8 bits.

Working. In Part C the 8 full-adder delays take 2.065 ns of logic, so one full-adder delay is about $2.065/8 \approx 0.258$ ns. The flip-flop overhead is 0.395 (CLK→Q) + 0.135 (setup) = 0.530 ns. For $N = 8$:

$$T_{min}(\text{8-bit}) \approx 0.530 + 16 \times 0.258 = 0.530 + 4.13 \approx 4.66 \text{ ns}$$

$$F_{max}(\text{8-bit}) \approx 1000/4.66 \approx 215 \text{ MHz} \quad \text{(vs 385 MHz for 4 bits; typical corner, pre-layout)}$$

The number of full adders grows about 4.7 times (quadratically in N), while the critical path grows only about 2 times in its logic part (linearly in N). T_{min} rises from 2.60 ns to about 4.66 ns, 1.8 times, because the flip-flop overhead stays the same. For area, scaling the logic by the number of building blocks, $(64 + 56)/(16 + 12) \approx 4.3$ times 405 μm^2 gives about 1740 μm^2; with 32 flip-flops (801 μm^2) the total is about 2540 μm^2 and the flip-flop share falls to about 32 %.